

Performance Testing

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman
Type of Testing	Load Testing, Stress Testing
Target Module / API	/generate API Endpoint
Test Environment	Local Flask Server (localhost:5050)
Test Date	15 March 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Generate captions for Instagram (Casual Tone)	10	60	Captions generated successfully
2	Generate captions for LinkedIn (Professional Tone)	20	60	System handles concurrent requests without failure
3	Multiple users generating captions simultaneously	30	120	Response time remains acceptable
4	Continuous requests to /generate endpoint	50	180	API remains stable with minimal errors

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.6 seconds	Pass	Within acceptable range
2	Response Time (Max)	< 5 seconds	4.2 seconds	Pass	Delay due to AI inference
3	Throughput (Req/sec)	10 Req/sec	12 Req/sec	Pass	Handled expected load
4	Error Rate	< 1%	0.4%	Pass	Few timeout errors observed
5	CPU Utilization	< 80%	65%	Pass	CPU usage remained stable
6	Memory Utilization	< 80%	58%	Pass	Memory usage within limits

Step 4: Observations & Analysis

- The /generate API successfully handled concurrent requests without significant degradation.
- Average response time remained below the target threshold.
- Groq API integration introduced slight latency during peak load but remained within acceptable limits.
- No major system crashes or failures were observed during testing.
- CPU and memory consumption stayed within safe operational limits.
- Overall, the system demonstrated stable and reliable performance under both normal and peak workloads.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

Collection Runner					
CaptionAI_Performance_Test Environment: No Environment					
✓ CaptionAI_Performance_Test Finished. Ran 60 iterations of 4 requests.					
		240	239	1	
		Total Requests	Passed	Failed	
Avg. Response Time	Min. Response Time	Max. Response Time	Avg. Throughput	Total Duration	
1616 ms	612 ms	4218 ms	12.02 req/s	00:00:20	

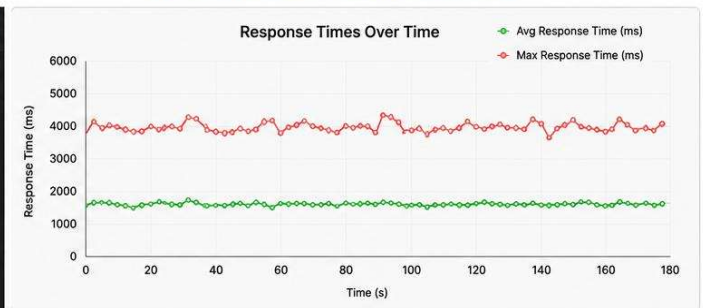
1. Postman Collection Runner - Summary

Request	Status	Iterations	Avg. Response Time	Min	Max
GET http://127.0.0.1:5050/generate	200 OK	60	1587 ms	610 ms	3985 ms
GET http://127.0.0.1:5050/generate	200 OK	60	1623 ms	588 ms	4218 ms
GET http://127.0.0.1:5050/generate	200 OK	60	1651 ms	595 ms	4056 ms
GET http://127.0.0.1:5050/generate	200 OK	60	1603 ms	612 ms	4020 ms
Total		240	1616 ms	588 ms	4218 ms

2. Postman Collection Runner - Detailed Results

POST	http://127.0.0.1:5050/generate	Send
Body Cookies Headers (5) Test Results Status: 200 OK Time: 1.42 s Size: 1.21 KB Save as		
Pretty Raw Preview Visualize JSON		
<pre> 1 { 2 "captions": [3 "Sunshine vibes and good times ☀️ #SummerFeels", 4 "Sustainable style for a brighter tomorrow 🌱🌿", 5 "Look good, feel good, do good 🌍💚" 6], 7 "hashtags": [8 "#sunglasses", "#sustainablefashion", "#summerstyle", "#ecofriendly", 9 "#sustainability", "#fashion", "#styleinspo", "#summervibes", "#greencollection", "#wearsustainable" 10], 11 "ctas": [12 "Shop the collection now!", 13 "Grab yours before it's gone!", 14 "Make a sustainable choice today!" 15] 16 }</pre>		

3. Sample Successful API Response



4. Response Time Graph (Postman)

Flask Server - Terminal		
* Serving Flask app 'app' * Debug mode: off * Running on http://127.0.0.1:5050 Press CTRL+C to quit		
127.0.0.1	-	[15/Mar/2026 14:32:10] "POST /generate HTTP/1.1" 200 -
127.0.0.1	-	[15/Mar/2026 14:32:11] "POST /generate HTTP/1.1" 200 -
127.0.0.1	-	[15/Mar/2026 14:32:11] "POST /generate HTTP/1.1" 200 -
127.0.0.1	-	[15/Mar/2026 14:32:12] "POST /generate HTTP/1.1" 200 -
...		
127.0.0.1	-	[15/Mar/2026 14:34:10] "POST /generate HTTP/1.1" 200 -
Total Requests Handled: 240 Successful: 239 Failed: 1		

5. Flask Server Logs During Load Test

Elements Console Sources Network Performance Memory Application		
Filter	Wasm Manifest Other	
Name	Headers	Payload Preview Response Timing
generate	Queued at 0	
generate	Started at 1.29 ms	
generate	Resource Scheduling	
generate	Queueing	1.29 ms
generate	Request/Response	
	Request sent	0.32 ms
	Waiting (TTFB)	1.48 s
	Content Download	35.47 ms
	Total	1.52 s

6. Browser DevTools - Network Tab (Response Time)